운영체제

6주차 – 교착상태

학습 목표

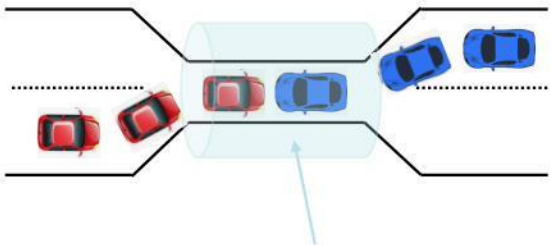
- **교착 상태를 이해한다.**
  - 정의와 발생원인
  - 발생 조건
  - 해결 방법

교착상태

교착 상태의 정의

- **교착 상태(deadlock)**
  - 2개 이상의 프로세스가 다른 프로세스의 작업이 끝나기만 기다리며 작업을 더 이상 진행하지 못하는 상태

- **교각 공유 예**
  - 교각을 **공유 자원(Resource)**에 비유
  - 교착상태가 발생하면 한쪽의 차가 후진을 해줘야 해결되는 상황
  - 기아상태에 빠질 수 있음
  - 대부분의 운영체제에서 교착상태를 완전히 예방해 주지는 못함



공유자원(임계구역, Critical Section)

- **아사 상태와 차이점**
  - **아사 현상(starvation)** : 운영체제가 잘못된 정책을 사용하여 특정 프로세스의 작업이 지연되는 문제
  - **교착 상태** : 여러 프로세스가 작업을 진행하다 보니 자연 발생적으로 일어나는 문제

교착 상태의 정의

- **시스템 자원**
  - 교착 상태는 다른 프로세스와 공유할 수 없는 자원을 사용할 때 발생

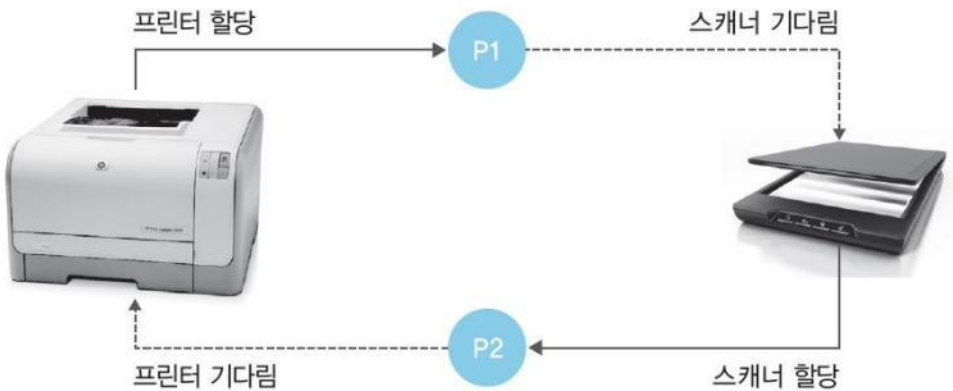


그림 6-3 시스템 자원 사용 도중에 발생한 교착 상태

교착 상태의 정의

- **잠금**
  - 교착 상태는 잠금을 사용할 때도 발생

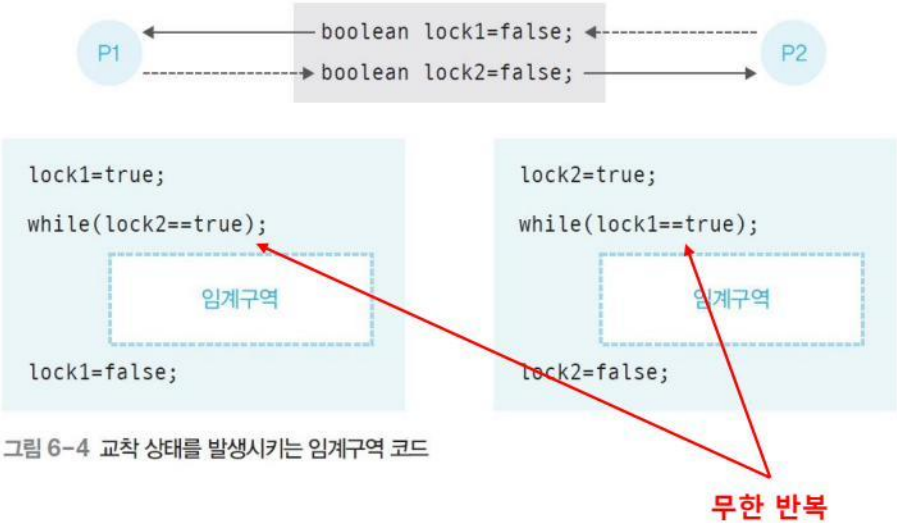


그림 6-4 교착 상태를 발생시키는 임계구역 코드

- **응용 프로그램**
  - 데이터베이스 같은 응용 프로그램에서도 교착 상태 발생
  - 데이터베이스는 데이터의 일관성을 유지하기 위해 잠금을 사용하는데, 이때 교착 상태가 발생할 수 있음

자원 할당 그래프

- **자원 할당 그래프(RAG : resource allocation graph)**
  - 프로세스가 어떤 자원을 사용 중이고 어떤 자원을 기다리고 있는지를 **방향성이 있는 그래프(directional graph)**로 표현한 것
  - 프로세스는 원으로, 자원은 사각형으로 표현

프로세스 P1이 자원 R1을 할당받음

프로세스 P1이 자원 R1을 기다림

그림 6-5 자원 할당 그래프

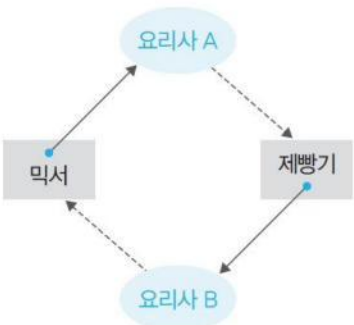


그림 6-7 요리사 문제의 자원 할당 그래프

자원 할당 그래프

- **다중 자원(multiple resource)**
  - 여러 프로세스가 하나의 자원을 동시에 사용하는 경우
  - 수용할 수 있는 프로세스 수를 사각형 안에 작은 동그라미로 표현

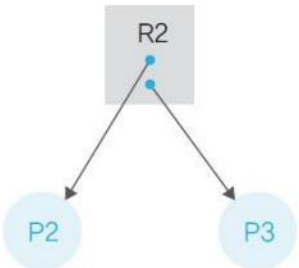
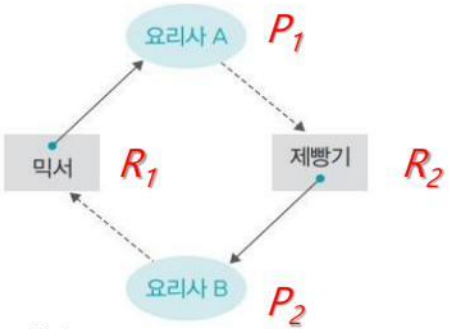


그림 6-6 2개의 프로세스를 수용할 수 있는 자원

자원 할당 그래프

- **교착상태를 수학적으로 기술하는 방법**
  - 그래프를 수식으로 표현
  - G = (V,E)
  - V : Vertex(정점)는 Process 집합(P)과 Resource 집합(R)으로 구성
    - $P = \{P_1, P_2, P_3, \ldots\}$
    - $R = \{R_1, R_2, R_3, \ldots\}$
  - E : Edge(간선)로 구성
    - 요청(Request Edge) : $P_i \rightarrow R_j$
    - 할당(Allocation Edge) : $R_j \rightarrow P_i$



$P = \{P_1, P_2\}$
$R = \{R_1, R_2\}$
$E = \{P_1 \rightarrow R_2,\ R_2 \rightarrow P_2,\ P_2 \rightarrow R_1,\ R_1 \rightarrow P_1\}$

자원 할당 그래프

- **식사하는 철학자 문제**
  - 왼쪽에 있는 포크를 잡은 뒤 오른쪽에 있는 포크를 잡아야만 식사 가능

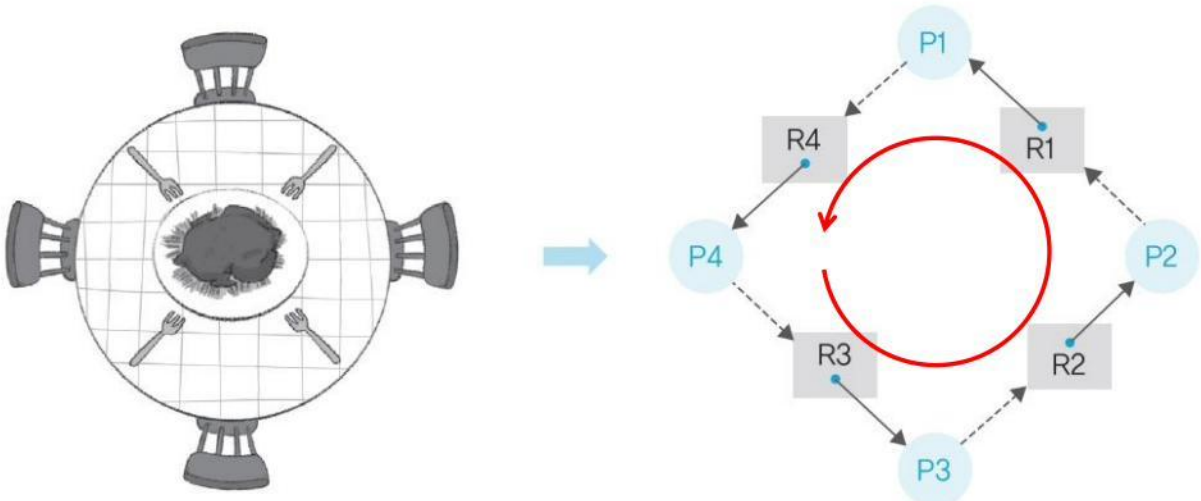


그림 6-8 식사하는 철학자 문제의 자원 할당 그래프

자원 할당 그래프

- **식사하는 철학자 문제에서 교착 상태가 발생하는 조건**

① 철학자들은 서로 포크를 공유할 수 없음

→ 자원을 공유하지 못하면 교착 상태 발생

② 각 철학자는 다른 철학자의 포크를 빼앗을 수 없음

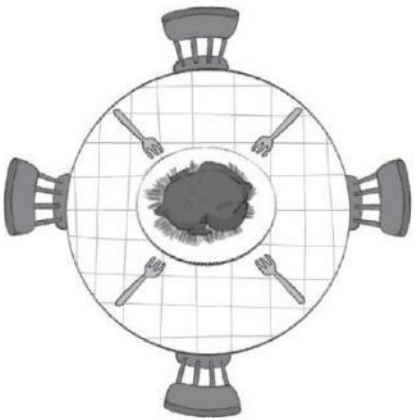
→ 자원을 놓을 때까지 기다려야 하므로 교착 상태 발생

③ 각 철학자는 왼쪽 포크를 잡은 채 오른쪽 포크를 기다림

→ 자원 하나를 잡은 상태에서 다른 자원을 기다리므로 교착 상태 발생

④ 자원 할당 그래프가 원형

→ 자원 요구 방향이 원이면 양보를 하지 않아 교착 상태 발생



교착 상태 필요조건

상호 배제, 비선점 조건
- 자원이 어떤 특징을 가지는지를 나타냄

점유와 대기, 원형 대기 조건
- 프로세스가 어떤 행위를 하고 있는지를 나타냄

- **교착 상태 필요 조건(necessary condition)**

  다음 4가지 조건 모두 만족해야 교착 상태 발생. 한가지라도 만족하지 않으면 교착 상태 발생하지 않음

  - **상호 배제**(mutual exclusion) : 한 프로세스가 사용하는 자원은 다른 프로세스와 공유할 수 없는 배타적인 자원이어야 함 → 배타적 자원 사용하면 교착 상태 발생
  - **비선점**(non-preemptive) : 한 프로세스가 사용 중인 자원은 중간에 다른 프로세스가 빼앗을 수 없는 비선점 자원이어야 함 → 빼앗을 수 없으면 공유 안되니 교착 상태 발생
  - **점유와 대기**(hold and wait) : 프로세스가 어떤 자원을 할당받은 상태에서 다른 자원을 기다리는 상태여야 함 → 자원을 점유하면서 다른 자원을 기다리면 교착 상태 발생
  - **원형 대기**(circular wait) : 점유와 대기를 하는 프로세스 간의 관계가 원을 이루어야 함 → 프로세스들이 서로 양보하지 않아 교착 상태 발생

식사하는 철학자 문제와 교착 상태 필요조건

- **식사하는 철학자 문제와 교착 상태 필요조건**

  - **상호 배제** : 포크는 한 사람이 사용하면 다른 사람이 사용할 수 없는 배타적 자원임
  - **비선점** : 철학자 중 어떤 사람의 힘이 월등하여 옆 사람의 포크를 빼앗을 수 없음
  - **점유와 대기** : 한 철학자가 두 자원(왼쪽 포크와 오른쪽 포크)을 다 점유하거나, 반대로 두 자원을 다 기다릴 수 없음
  - **원형 대기** : 철학자들이 식사하는 둥근 식탁은 선후 관계를 결정할 수 없어 문제가 계속 맴돈다는 의미(사각형 식탁에서 한 줄로 앉아서 식사를 하면 교착 상태가 발생하지 않음)

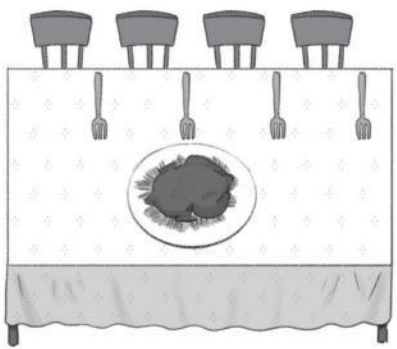


그림 6-10 사각형 식탁에서 일렬로 식사하는 철학자 문제

교착 상태 해결

- **교착 상태 해결 방법**
  - **교착 상태 예방**(prevention) : 교착 상태를 유발하는 네 가지 조건이 발생하지 않도록 무력화하는 방식으로 교착상태 조건 4가지에 대하여 각각의 방법이 존재 함. (<u>하나라도 조건을 만족하지 않으면 교착상태 발생 X</u>)
    - 상호배제, 비선점, 점유와 대기, 원형대기
  - **교착 상태 회피**(avoidance) : 교착상태가 발생하지 않도록 자원 할당량을 조절하여 교착 상태를 회피하는 방식
    - 시스템의 상태를 계속 감시 → 안전한 상태(Safe State)로 유지
  - **교착 상태 검출**(detection)**과 회복**(recovery) : 교착 상태 검출은 어떤 제약을 가하지 않고 자원 할당 그래프를 모니터링하면서 교착 상태가 발생하는지 살펴보는 방식으로 만약 교착 상태가 발생하면 교착 상태 회복 단계가 진행됨
  - **교착상태 무시**(Ignore) : 교착상태가 드물게 발생하는 경우 예방, 검출과 같은 지속된 오버헤드 및 성능 저하를 발생하는 것보다 재부팅하는 것이 더 좋을 수 있음

교착 상태 예방

- **상호 배제 예방**
  - 시스템 내에 있는 상호 배타적인 모든 자원, 즉 독점적으로 사용할 수 있는 자원을 없애는 방법 → 자원 공유를 허용
  - 현실적으로는 모든 자원을 공유할 수 없으며 상호 배제를 적용하여 보호해야 하는 자원이 있음
  - 상호 배제를 무력화하는 것은 사실상 어려움

- **비선점 예방**
  - 모든 자원을 빼앗을 수 있도록 만드는 방법 → 선점(preemptive)을 허용
  - 그러나 아사 현상을 일으켜 비선점 조건을 무력화하기는 어려움

교착 상태 예방

- **점유와 대기 예방**
  - 프로세스가 자원을 점유한 상태에서 다른 자원을 기다리지 못하게 하는 방법
  - '**전부 할당하거나 아니면 아예 할당하지 않는**' 방식을 적용
    - 전부 할당할 경우, 자원이 필요 없음에도 할당(자원 낭비)되거나 무한 대기가 발생할 수 있음
  - 자원이 아닌 프로세스의 자원 사용 방식을 변화시켜 교착 상태를 처리한다는 점에서 의미가 있음

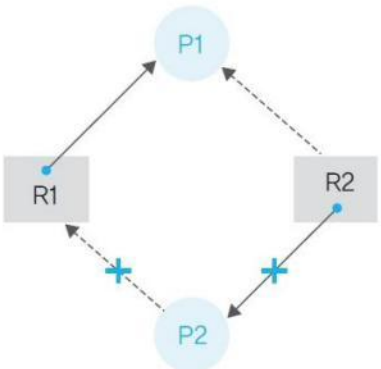


그림 6-11 전부 할당하거나 아예 할당하지 않는 방식

교착 상태 예방

- **점유와 대기 예방의 단점**
  - 프로세스가 자신이 사용하는 모든 자원을 자세히 알기 어려움
  - 자원의 활용성이 떨어짐
  - 많은 자원을 사용하는 프로세스가 적은 자원을 사용하는 프로세스보다 불리함
  - 결국 일괄 작업 방식으로 동작

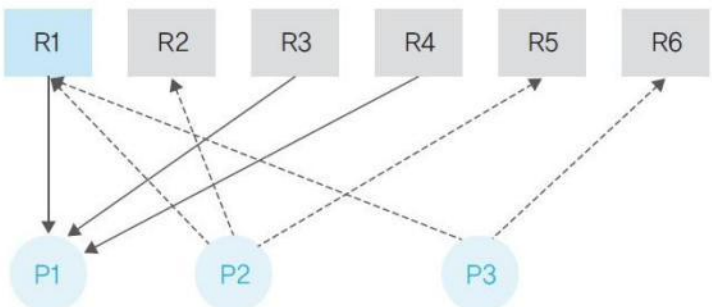


그림 6-12 필요한 자원을 모두 할당한 후 실행하는 방식

교착 상태 예방

- **원형 대기 예방**
  - 점유와 대기를 하는 프로세스들이 원형을 이루지 못하도록 막는 방법
  - 모든 자원에 숫자를 부여하고 숫자가 큰 방향으로만 자원을 할당하는 것

[예] 마우스를 할당받은 상태에서 프린터를 할당받을 수는 있지만,
프린터를 할당받은 상태에서는 마우스나 하드디스크를 할당받을 수 없음

그림 6-13 자원에 번호 매김

교착 상태 예방

- **원형 대기 예방과 교착 상태 해결**
  - 프로세스 P2는 자원을 할당받을 수 없어 강제 종료되고 프로세스 P1은 정상적으로 실행

- **원형 대기 예방의 단점**
  - 프로세스 작업 진행에 유연성이 떨어짐
  - 자원의 번호를 어떻게 부여할 것인지가 문제

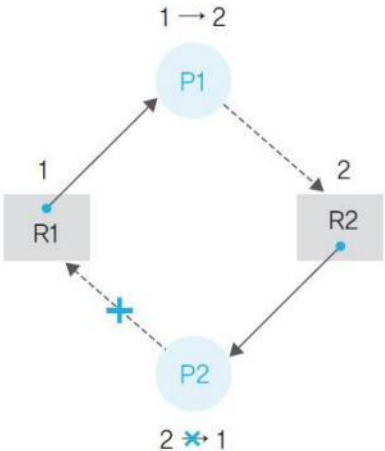


그림 6-14 원형 대기 조건의 부정

교착 상태 예방

- **교착 상태 예방 정리**
  - 교착 상태를 유발하는 **네 가지 조건**이 일어나지 않도록 제약을 가하는 방법
  - 자원을 보호해야 하므로 **상호 배제**와 **비선점** 예방은 사용하기 어려움
  - **점유와 대기, 원형 대기** 예방은 프로세스 작업 방식을 제한하고 자원을 낭비하기 때문에 사용할 수 없음

교착 상태 회피

- **교착 상태 회피의 개념**
  - 프로세스에 자원을 할당할 때 어느 수준 이상의 자원을 나누어주면 교착 상태가 발생하는지 파악하여 그 수준 이하로 자원을 나누어주는 방법
  - 교착 상태가 발생하지 않는 범위 내에서만 자원을 할당하고, 교착 상태가 발생하는 범위에 있으면 프로세스를 대기시킴
    - 시스템을 계속 감시하다가 교착상태의 가능성이 있을 경우 요청 보류
  - 즉, 할당되는 자원의 수를 조절하여 교착 상태를 피함

교착 상태 회피

- **안정 상태와 불안정 상태**
  - 교착 상태 회피는 자원 총수와 현재 할당된 자원 수를 기준으로 시스템을 안정 상태(safe state)와 불안정 상태(unsafe state)로 나누고 시스템이 안정 상태를 유지하도록 자원을 할당
    - 안정 상태 : 모든 프로세스가 정상적으로 종료가 가능한 상태
    - 불안정 상태 : 교착 상태가 될 가능성이 있음(항상 교착상태는 아님)
  - 할당된 자원이 적으면 안정 상태가 크고, 할당된 자원이 늘어날수록 불안정 상태 커짐
  - 교착 상태는 불안정 상태의 일부분이며, 불안정 상태 커질수록 교착 상태 발생 가능성 높아짐
  - 교착 상태 회피는 안정 상태를 유지할 수 있는 범위 내에서 자원을 할당해 교착 상태를 피함

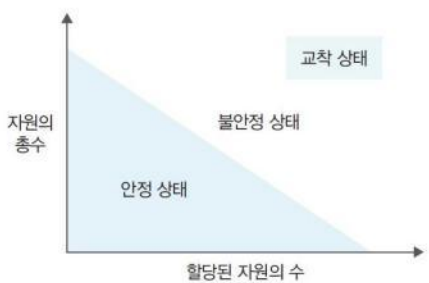


그림 6-15 안정 상태와 불안정 상태

교착 상태 회피

- **은행원 알고리즘**
  - 교착 상태 회피를 구현하는 대표적인 알고리즘 (다익스트라(Dijkstra)가 제안)
  - 은행이 대출을 해주는 방식, 즉 대출 금액이 대출 가능한 범위 내이면(안정 상태이면) 허용되지만 그렇지 않으면 거부되는 것과 유사한 방식
    - 프로세스가 요구한 자원의 수가 현재 가용한(남은) 자원의 수보다 작을 때 할당

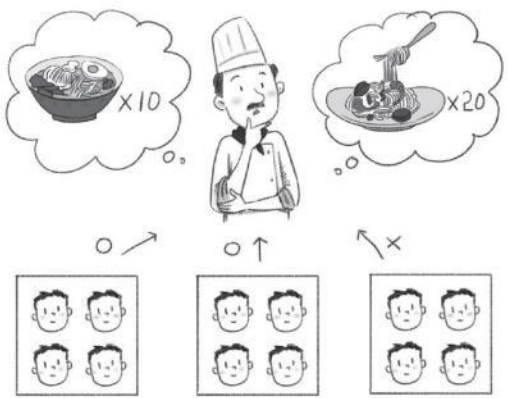


그림 6-16 은행원 알고리즘

교착 상태 회피

표 6-1 은행원 알고리즘의 변수

변수	설명
Total(전체 자원)	시스템 내 전체 자원의 수
Available(가용 자원)	시스템 내 현재 사용할 수 있는 자원의 수 가용 자원 = 전체 자원 – 모든 프로세스의 할당 자원
Max(최대 자원)	각 프로세스가 선언한 최대 자원의 수
Allocation(할당 자원)	각 프로세스에 현재 할당된 자원의 수
Expect(기대 자원)	각 프로세스가 앞으로 사용할 자원의 수 기대 자원 = 최대 자원 – 할당 자원

교착 상태 회피

- **은행원 알고리즘에서 자원 할당 기준**
  - 각 프로세스의 기대 자원과 비교하여 가용 자원이 하나라도 크거나 같으면 자원을 할당
  - 가용 자원이 어떤 기대 자원보다 크지 않으면 할당하지 않음

정의 6-1 안정 상태

각 프로세스의 기대 자원과 비교하여 가용 자원이 크거나 같은 경우가 한 번 이상인 경우다.

교착 상태 회피

- **안정 상태의 예**
  - 할당 순서 : P2 → P1 → P3

Total=14 Available=2

Process	Max	Allocation	Expect
P1	5	2	3
P2	6	4	2
P3	10	6	4

그림 6-17 은행원 알고리즘(안정 상태)

- **불안정 상태의 예**
  - 할당 불가

Total=14 Available=1

Process	Max	Allocation	Expect
P1	7	3	4
P2	6	4	2
P3	10	6	4

그림 6-18 은행원 알고리즘(불안정 상태)

교착 상태 회피

- **교착 상태 회피의 문제점**
  - 시스템을 항상 감시하고 있어야 하므로, 오버헤드 발생
  - 프로세스가 자신이 사용할 모든 자원을 미리 선언해야 함(미리 알고 있어야 함)
  - 시스템의 전체 **자원 수** 및 **프로세스 수**가 고정적이어야 함
  - 사용되지 않는 여유 자원이 있어야 하는 등 자원 활용률이 낮음

교착 상태 검출

- **교착 상태 검출의 개념**
  - 운영체제가 프로세스의 작업을 관찰하면서 교착 상태 발생 여부를 계속 주시하는 방식
  - 교착 상태가 발견되면 이를 해결하기 위해 교착 상태 회복 단계를 밟음

- **타임아웃을 이용한 교착 상태 검출**
  - 일정 시간 동안 작업이 진행되지 않은 프로세스를 교착 상태가 발생한 것으로 간주하여 처리
  - 교착 상태가 자주 발생하지 않을 것이라는 가정하에 사용하는 것으로, 특별한 알고리즘이 없어 쉽게 구현
  - 타임아웃이 되면 프로세스 종료

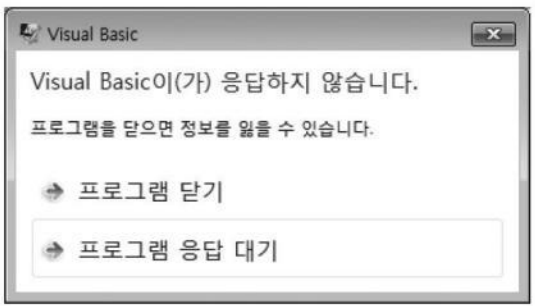


그림 6-19 타임아웃을 이용하는 방법의 예

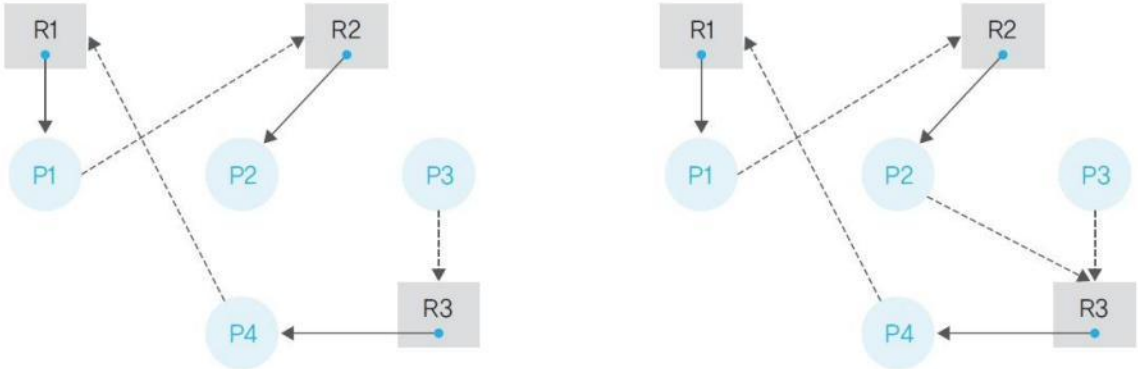
교착 상태 검출

- **데이터베이스에서 타임아웃의 문제**
  - 데이터베이스에서 타임아웃으로 프로세스가 종료되면 일부 데이터의 일관성이 깨질 수 있음
  - 데이터의 일관성이 깨지는 문제를 해결하기 위해 체크포인트와 롤백 사용
    - **체크포인트**(check point) : 작업 중 문제 발생하면 저장된 상태로 돌아오기 위한 표시
    - **롤백**(roll back) : 작업 중 문제 발생하여 과거의 체크포인트로 되돌아가는 것

그림 6-20 체크포인트와 롤백

교착 상태 검출

- **자원 할당 그래프를 이용한 교착 상태 검출**
  - 단일 자원을 사용하는 경우 자원 할당 그래프에 **사이클** 있으면 교착 상태



(a) 교착 상태가 없는 자원 할당 그래프

(b) 교착 상태가 있는 자원 할당 그래프

그림 6-22 자원 할당 그래프와 교착 상태

교착 상태 회복

- **교착 상태 회복**
  - 교착 상태가 검출된 후 교착 상태를 푸는 후속 작업을 하는 것
  - 교착 상태 회복 단계에서는 교착 상태를 유발한 프로세스를 강제로 종료

  - 프로세스를 강제로 종료하는 방법

① 교착 상태를 일으킨 모든 프로세스를 **동시에 종료**

② 교착 상태를 일으킨 프로세스 중 하나를 골라 **순서대로 종료**

  - **우선순위가 낮은** 프로세스를 먼저 종료
  - 우선순위가 같은 경우 **작업 시간이 짧은** 프로세스를 먼저 종료
  - 위의 두 조건이 같은 경우 **자원을 많이 사용하는** 프로세스를 먼저 종료

교착 상태 무시

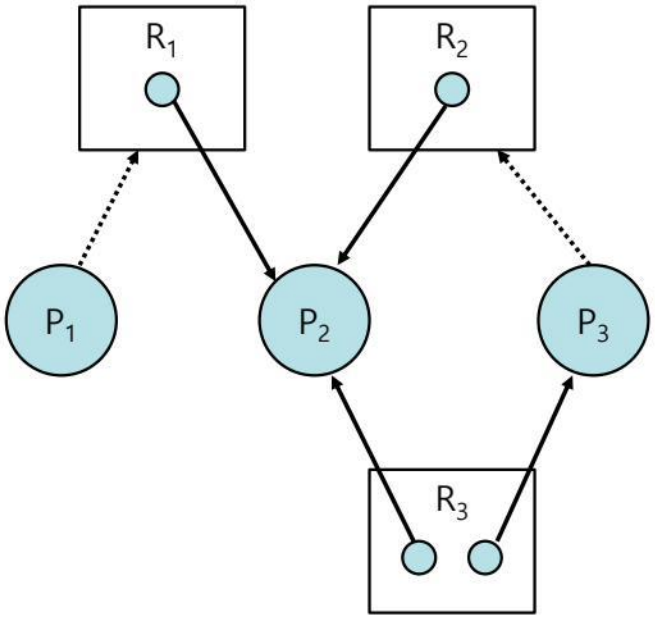
- **교착상태에 대해 아무런 대비책 없이 발생하도록 내버려 둠**
  - 교착 상태 발생확률이 낮은 경우
  - 교착상태 예방, 회피, 탐지에서 오버헤드가 발생하므로
  - 교착상태가 발생하면, 재시작 혹은 강제 종료(대부분의 OS에서 사용)
    - 중요한 데이터를 다루지 않는 환경에서 가능
    - 데이터 손실의 위험이 있음

  ※ 핵 시스템, 원자력 시스템, 비행기 등과 같은 경성 실시간 시스템 (hard-real time)에서는 적합하지 않음

- **타조(Ostrich) 알고리즘**
  - 타조가 위험이 발생하면 머리를 땅에 박고 무시함
  - 현실 부정, 현실 도피
  - **교착상태를 무시**

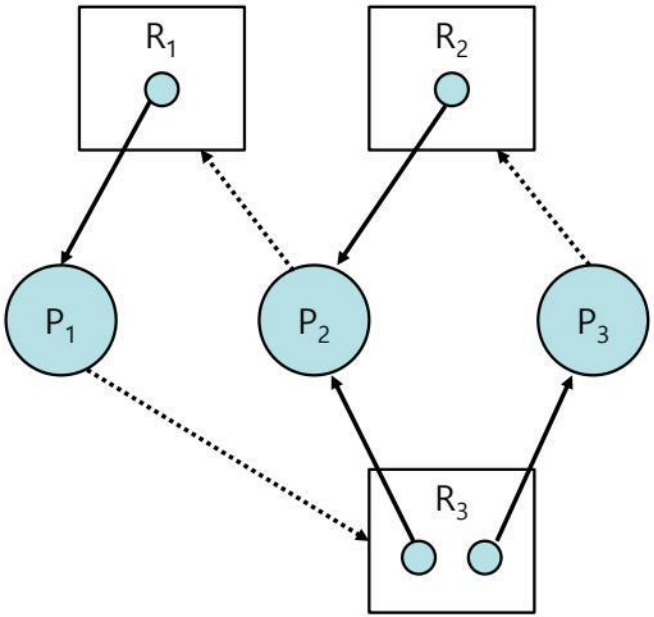
교착상태 분석 (1/3)

- **그래프 예 (1/3)**
  - 교착상태 발생?



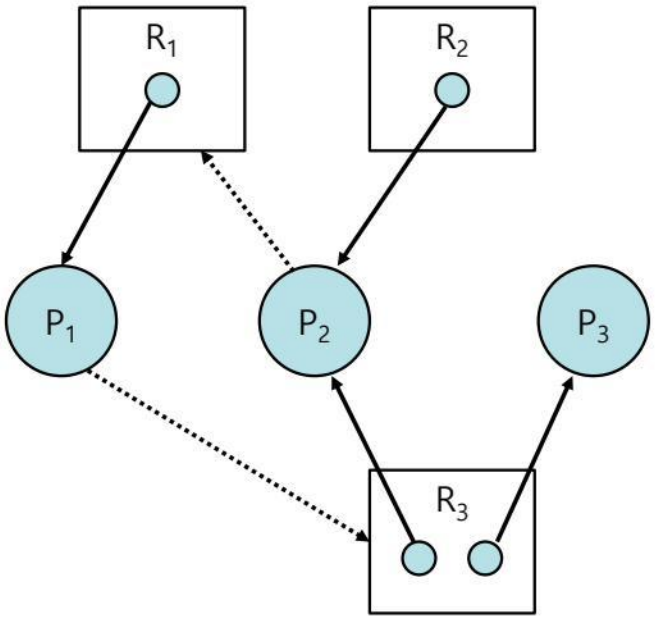
교착상태 분석 (2/3)

- **그래프 예 (2/3)**
  - 교착상태 발생?



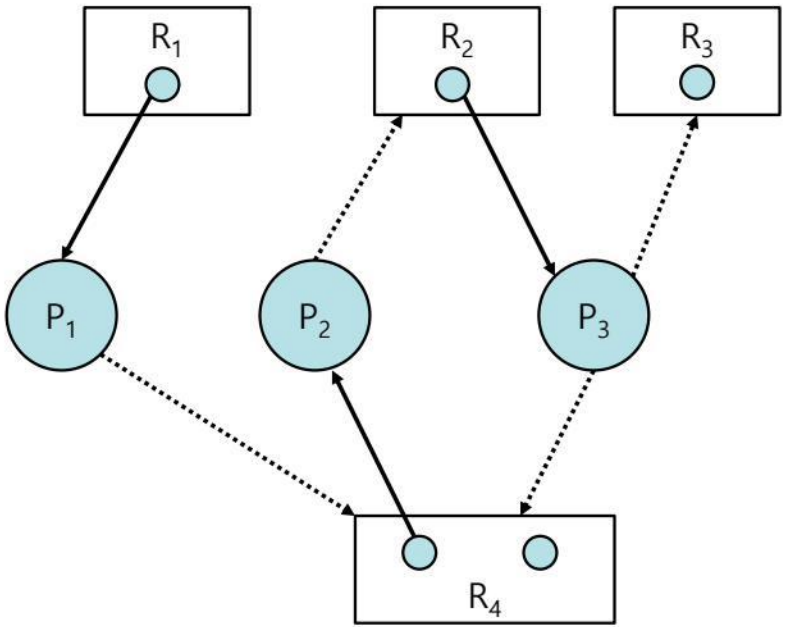
교착상태 분석 (3/3)

- **그래프 예 (3/3)**
  - 교착상태 발생?



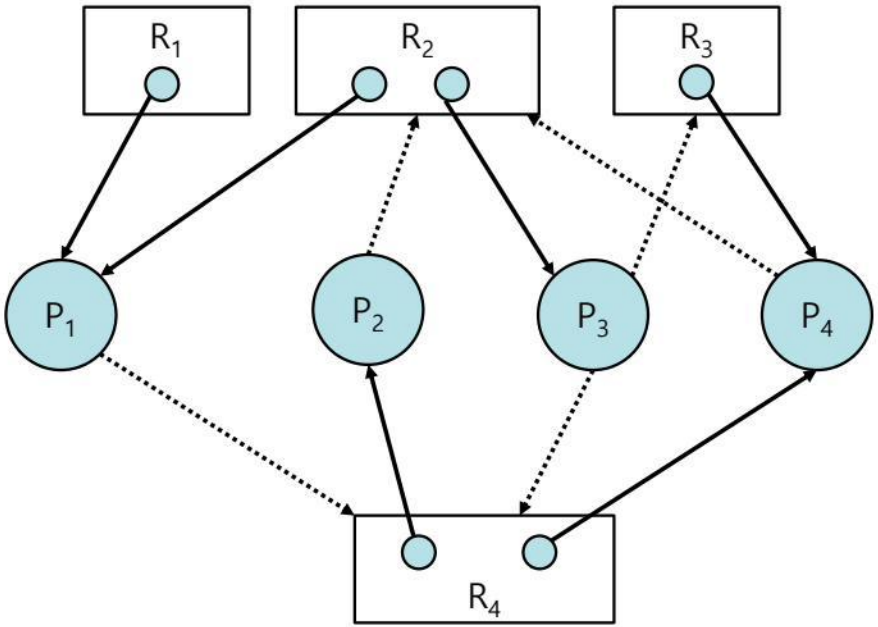
※ 그래프 심화 1

- **그래프 문제 #1**
  - 교착상태 발생?
  - 미발생시 실행 순서는?



※ 그래프 심화 2

- **그래프 문제 #2**
  - 교착상태 발생?
  - 미발생시 실행 순서는?



[실습] 임계구역 1 (1/4)

- **임계 구역 데이터의 동시 접근 실습 (1/4)**

```c
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <ctype.h>

void* mythread_r(void* ptr);
void* mythread_w(void* ptr);

int
main(int argc, char* argv[])
{
    pthread_t thread1, thread2;
    int nValue = 0;

    // start the threads
    pthread_create(&thread1, NULL, *mythread_r, (void*)&nValue);
    pthread_create(&thread2, NULL, *mythread_w, (void*)&nValue);

    // wait for threads to finish
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    return 0;
}
```

[실습] 임계구역 1 (2/4)

- **임계 구역 데이터의 동시 접근 실습 (2/4)**

```c
void*
mythread_r(void* p_Value)
{
    int* p_nValue = (int*)p_Value;

    do
    {
        if (*p_nValue % 2 == 0)
        {
            usleep(500000);
            printf("%d is even number!!₩n", *p_nValue);
        }
        else
        {
            usleep(500000);
            printf("%d is odd number!!₩n", *p_nValue);
        }

        usleep(500000);
    } while (1);

    return  p_nValue;
}
```

```c
void*
mythread_w(void* p_Value)
{
    int* p_nValue = (int*)p_Value;

    do
    {
        (*p_nValue)++;

        usleep(500000);
    } while (1);

    return  p_nValue;
}
```

[실습] 임계구역 1 (3/4)

- **임계 구역 데이터의 동시 접근 실습 (3/4)**

①

```c
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <ctype.h>

void* mythread_r(void* ptr);
void* mythread_w(void* ptr);
```

②

```c
int
main(int argc, char* argv[])
{
    pthread_t thread1, thread2;
    int nValue = 0;

    // start the threads
    pthread_create(&thread1, NULL, *mythread_r, (void*)&nValue);
    pthread_create(&thread2, NULL, *mythread_w, (void*)&nValue);

    // wait for threads to finish
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    return 0;
}
```

③

```c
void*
mythread_r(void* p_Value)
{
    int* p_nValue = (int*)p_Value;

    do
    {
        if (*p_nValue % 2 == 0)
        {
            usleep(500000);
            printf("%d is even number!!\n", *p_nValue);
        }
        else
        {
            usleep(500000);
            printf("%d is odd number!!\n", *p_nValue);
        }

        usleep(500000);
    } while (1);

    return  p_nValue;
}
```

④

```c
void*
mythread_w(void* p_Value)
{
    int* p_nValue = (int*)p_Value;

    do
    {
        (*p_nValue)++;

        usleep(500000);
    } while (1);

    return  p_nValue;
}
```

[실습] 임계구역 1 (4/4)

- **임계 구역 데이터의 동시 접근 실습 (4/4)**
  - 코드 빌드 및 실행
    - gcc critial_section.c –o critial_section
    - ./critical_section

```
2 is odd number!!
4 is odd number!!
6 is odd number!!
8 is odd number!!
9 is even number!!
11 is even number!!
13 is even number!!
15 is even number!!
17 is even number!!
19 is even number!!
22 is even number!!
23 is odd number!!
26 is odd number!!
27 is odd number!!
30 is odd number!!
^C
[kali@kali:05_1]$
```

대체적으로 우리가 원하는 값이 맞음?

[실습] 임계구역 2 (1/3)

- **세마포어 적용 후 실습 1의 문제 해결 (1/3)**
  - 코드 작성

```c
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <ctype.h>
#include <semaphore.h>

void* mythread_r(void* ptr);
void* mythread_w(void* ptr);

sem_t g_mutex;

int
main(int argc, char* argv[])
{
    pthread_t thread1, thread2;
    int nValue = 0;

    sem_init(&g_mutex, 0, 1); // initialize g_mutex

    // start the threads
    pthread_create(&thread1, NULL, *mythread_r, (void*)&nValue);
    pthread_create(&thread2, NULL, *mythread_w, (void*)&nValue);

    // wait for threads to finish
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    sem_destroy(&g_mutex); // destroy g_mutex

    return 0;
}
```

코드 추가

[실습] 임계구역 2 (2/3)

- **세마포어 적용 후 실습 1의 문제 해결 (2/3)**
  - 코드 작성

```c
void*
mythread_r(void* p_Value)
{
    int* p_nValue = (int*)p_Value;

    do
    {
        sem_wait(&g_mutex); // wait and lock g_mutex
        if (*p_nValue % 2 == 0)
        {
            usleep(500000);
            printf("%d is even number!!\n", *p_nValue);
        }
        else
        {
            usleep(500000);
            printf("%d is odd number!!\n", *p_nValue);
        }
        sem_post(&g_mutex); // release g_mutex

        usleep(500000);
    } while (1);

    return  p_nValue;
}
```

코드 추가

```c
void*
mythread_w(void* p_Value)
{
    int* p_nValue = (int*)p_Value;

    do
    {
        sem_wait(&g_mutex);// wait and lock g_mutex
        (*p_nValue)++;
        sem_post(&g_mutex); // release g_mutex

        usleep(500000);
    } while (1);

    return  p_nValue;
}
```

[실습] 임계구역 2 (3/3)

- **세마포어 적용 후 실습 1의 문제 해결 (3/3)**
  - 빌드 및 실행
    - gcc semaphore.c –o semaphore
    - ./semaphore

```
0 is even number!!
2 is even number!!
4 is even number!!
6 is even number!!
8 is even number!!
10 is even number!!
11 is odd number!!
12 is even number!!
14 is even number!!
15 is odd number!!
17 is odd number!!
^C
```

해결 확인